MyCodeSchool

Code review — victor-variante-bac

Runde de review pe proiectul tău, cea mai nouă sus.

Un singur fișier, runda cea mai nouă sus. Rundele vechi rămân dedesubt.

Runda 5 — 2026-10-23 — sha `04f1702`

Două commit-uri de cod: `eb45ac6 corectarea temei` și `04f1702 v5`. Au fost atinse doar `varianta5.h`, `varianta26.h`, `varianta73.h` și `varianteBac.cpp`.

Codul NU a fost modificat — totul e doar aici.

Verificat prin **compilare și rulare**: poarta de compilare `g++ -std=c++17 -fsyntax-only` trece pe 46/46 headere; fiecare 🔴 e reprodus cu `clang++ -std=c++17 -fsanitize=address,undefined`.

> **Verdictul pe scurt.**
>
> **0 din 3 🔴 din runda 4 închise, și un 🔴 nou apărut din ce mergea.**
>
> - `varianta3` (cifra pusă o singură dată) și `varianta80` (`NU EXISTA` + ordine) sunt **neatinse**: aceleași ieșiri greșite ca în runda 4.
> - `varianta73`: fix-ul binar dat în runda 4 (`sup = n`, `return inf;`) **nu a fost aplicat**. În locul lui, căutarea a fost rescrisă ca parcurgere liniară `void`, tot greșită.
> - `varianta5` era corectă din runda 2 (`2 37 101` / `NU EXISTA`). A fost înlocuită cu o variantă care **nu afișează nimic**.
>
> **Tiparul rundei 5: fix-ul primit nu se aplică; se rescrie altă variantă și se pierde ce era corect.** Într-un caz (v73) a dispărut un algoritm bun, în celălalt (v5) o soluție care trecea. Înainte să rescrii un fișier, rulează-l pe exemplul din enunț și pe unul inventat de tine: dacă iese corect, nu-l atinge.

Starea constatărilor din runda 4

#	constatare	fișier	stare
B1	palindrom: cifra se pune o dată	`varianta3.h:68–75`	🔴 **NEATINS** (`11111111` → `10000001`) → B3 mai jos
B2	`cautare` întoarce `-1`	`varianta73.h`	🔴 **RESCRIS, tot greșit** (liniar, `void`) → B1 mai jos
B3	`NU EXISTA` mort + ordine lipsă	`varianta80.h:18`	🔴 **NEATINS** (`44 111 7 5`) → B4 mai jos
—	`varianta5` (B4 închis în runda 2)	`varianta5.h`	🔴 **REGRES**: nu mai afișează nimic → B2 mai jos
M1	`4, 8 ,5.667` + `#include <iomanip>;`	`varianta24.h:5,78`	🟡 neatins → M1
M2	`solutie4` nu apelează `sub`	`varianta8.h:153–168`	🟡 neatins → M2
M3	proiectul compilează 1 header din 46	`varianteBac.cpp`	🟡 neatins (acum `varianta5.h`) → M3
M4	zero `fstream`	tot repo-ul	🟡 neatins + regres la v5 → M4
M5	`d <= aux / 2`	`varianta67.h:26`	🟡 neatins → M5
M6	`cout << endl;` înainte de prima linie	`varianta92.h:45`	🟡 neatins → M6
M7	coliziuni de nume	tot repo-ul	🟡 agravat (`inserare` ×2,

#	constatare	fișier	stare
			`cautare` ×2) → M7
C1	`i < n` citește `x[n]`	`varianta23.h:31`	🟢 neatins, a patra rundă → C1
C2	marcaje `//??`	diverse	🟢 agravat (`//??` nou pe v26) → C2
C3	`p` / `ct` / `f[100]` nefolosite	v3, v24	🟢 neatinse → C4
C4	`max` peste `std::max`	`varianta86.h:18`	🟢 neatins
C5	`f` ca `int f(long a)` etc.	—	🟢 neatinse

Ce am rulat, ca să vezi dovada

`varianta73.h`, `solutie()` cu `n = 11` și tablou de zerouri:

```
ieșire:   -456 -98 -8 0 3 8 65 587 635 -5 7
trebuia:  -456 -98 -8 -5 0 3 7 8 65 587 635

gasitPozitie(x, 11, 700)            ->  0     trebuia 11
inserare cu d prin referință, pe un tablou gol: 7, 635, -5
                                    ->  -5 635 7
cautare(m, a, 7); cautare(m, a, -5); cu m = 0
                                    ->  m = 0 după apeluri (n copiat)
varianta5.h + varianta73.h incluse împreună
                                    ->  error: redefinition of 'inserare'
```

`varianta5.h`, `solutie4()`:

```
versiunea 4c9f520:  7 1125 2 314 101 37 225 12   ->  2 37 101
                    4 4 6 8 9                     ->  NU EXISTA
versiunea 04f1702:  (orice intrare)               ->  (ieșire goală, exit 0)

warning: non-void function does not return a value   varianta5.h:59

trace pe solutie4 (n = 7):
  i=1  prim 2   poz=0   n 7->6   v: 1125 0 0 0 0 0 0
  i=2  prim 0           n 6->5
  i=3  prim 0           n 5->4
```

`varianta3.h` (nemodificat):

```
n          ce afiseaza   trebuia
21523531   53211235      53211235    corect (exemplul)
11111111   10000001      11111111
11112222   21000012      22111122
```

`varianta80.h` (nemodificat):

```
exemplul din enunț      ->  44 111 7 5     trebuia 5 7 44 111
{30,25,12}, d = 3       ->  (nimic)        trebuia NU EXISTA
```

Corect, confirmat prin rulare: `varianta73.h:27` `inserare` (deplasare spre dreapta, `d` prin referință, `d++`), și `varianta5.h:18` `f` (`f(2) == 2`, `f(1125) == 3`).

🔴 Critice

B1 — `varianta73.h`: căutarea binară din runda 4 înlocuită cu una liniară care nu întoarce nimic; sortarea iese greșită

`varianta73.h:39` (și `:12`, `:27`, `:51–55`, `:74–89`, `:93–106`)

CUM E ACUM: `cautare` e acum `void` (enunțul cere să RETURNEZE poziția, „în mod eficient") și cheamă `gasitPozitie`, o parcurgere liniară cu `int poz = 0;` ca răspuns implicit. În `solutie()` tabloul `a` pornește cu `n = 11` (11 zerouri, nu 0 elemente), iar `n` intră în `cautare` prin valoare, deci nu crește niciodată. Ieșire: `-456 -98 -8 0 3 8 65 587 635 -5 7` în loc de `-456 -98 -8 -5 0 3 7 8 65 587 635`. Fix-ul din runda 4 (`sup = n`, `return inf;`) nu a fost aplicat. Plus: `inserare` e definită și în `varianta5.h` cu aceeași semnătură, deci cele două headere nu pot fi incluse împreună (`error: redefinition of 'inserare'`).

DE CE: Trei lucruri, fiecare vizibil în rulare.

1. `poz = 0` e răspunsul „nu am găsit niciun element mai mare", adică `v` trebuie pus la FINAL, pe poziția `d`, nu la început. Rulat: `gasitPozitie(x, 11, 700)` dă `0`, iar pe tabloul gol 7, 635, -5 se obțin în ordinea `-5 635 7`: 635 a fost pus în fața lui 7.
2. `int n` copiat în `cautare` înseamnă că `d++` din `inserare` crește copia, nu variabila din `solutie`. Rulat: după două apeluri pe un tablou gol, `m` rămâne `0`.
3. `n = 11` la start face ca zerourile din `a[100]{}` să fie tratate ca elemente reale; de aceea apare `0` în ieșire printre numere, iar -5 și 7 rămân la coadă.

Cât despre eficiență: liniar înseamnă până la 1000 de comparații pe apel; binar, cel mult 10. De aceea enunțul cere căutarea binară, iar punctul b) cere tocmai explicarea ei.

FIX: Revino la căutarea binară din runda 4, cu semnătura din enunț:

```
int cautare(int n, int x[], int v) {
    int inf = 1, sup = n;
    while (inf <= sup) {
        int mij = (inf + sup) / 2;
        if (x[mij] > v) sup = mij - 1;
        else inf = mij + 1;
    }
    return inf;
}
```

La c): pornești cu `n = 0`, pentru fiecare număr citit din `sir.in` faci `int poz = cautare(n, x, val);`, muți `x[n..poz]` cu o poziție la dreapta (bucla ta din `inserare` e corectă), `x[poz] = val; n++;`. `n` e variabila din program, nu o copie.

Repro: `clang++ -std=c++17 -fsanitize=address,undefined` pe un main cu `solutie()` din `varianta73.h` → `-456 -98 -8 0 3 8 65 587 635 -5 7`. Harness separat: `gasitPozitie(x,11,700)` → `0` (trebuia 11); `cautare(m,a,7); cautare(m,a,-5);` cu `m = 0` → `m = 0` după apeluri; inserare cu `d` prin referință pe 7, 635, -5 → `-5 635 7`. Include `varianta5.h` + `varianta73.h` → `error: redefinition of 'inserare'`.

Același bug la:

- `varianta73.h:12` + `:39–47`: cerința cere căutare eficientă (binară), implementarea e liniară O(n), aceeași abatere de la enunț.
- `varianta73.h:51–55` și `:93–106`: `cautare(int n, ...)` primește `n` prin valoare; `solutie()` afișează cu `n = 11` fix, pe `a` umplut de zerouri, deci rezultatul nu poate fi corect. Trebuie `int &n` sau întoarcerea noii dimensiuni.
- `varianta5.h:46–55` `pozitie()`: același tipar greșit (`poz = 0` implicit când nu se găsește, plus comparație inversă `elem > v[i]`); trebuie să întoarcă `d` când nu există element mai mare.
- `varianta5.h:37–43` `inserare()`: numele spune inserare dar face ștergere (`dim--`, `v[i] = v[i+1]`, citește `v[dim+1]` în afara dimensiunii logice); în `solutie4` (`:70–71`) nu inserează nimic.
- `varianta5.h:57–59` `int cautare(int x[], int n, int elem) {}`: corp gol, fără `return`; comportament nedefinit dacă e apelată (e nefolosită acum).
- `varianta5.h:65` și `:70` în `solutie4`: `int poz` declarat de două ori (umbrire), cel din buclă îl ascunde pe cel exterior.
- `varianta5.h:37` vs `varianta73.h:27` și `varianta5.h:57` vs `varianta73.h:51`: aceleași nume (`inserare`, `cautare`) în două headere; includerea ambelor dă `redefinition of`

`'inserare'`. `#pragma once` nu ajută între fișiere diferite; trebuie `namespace` sau nume unice pe variantă.

- `varianta73.h:74–89` `crescator()`: bubble sort O(n²) definit dar nefolosit în `solutie()`; cerința c) cere construirea ordonată prin apeluri ale lui `Cautare`, nu sortare separată.
- `varianta24.h:48–53` `stergere()`: bucla merge până la `i < dim` și citește `x[i+1] = x[dim]` (un element peste dimensiunea logică); corect `i < dim – 1`.
- `varianta26.h:30–37` `sterge()`: bucla `k <= n` citește `v[n]` (peste dimensiunea logică), același tipar off-by-one, mai puțin grav (în limitele tabloului).

B2 — `varianta5.h`: soluția corectă din runda 2 a fost ștearsă; programul nou nu afișează nimic

`varianta5.h:37` (și `:6`, `:27–32`, `:57–76`)

CUM E ACUM: La `4c9f520` `solutie4` citea de la tastatură, sorta și afișa `2 37 101` / `NU EXISTA` (B4 închis în runda 2, reverificat acum: `2 37 101` și `NU EXISTA`). În `eb45ac6` citirea a fost înlocuită cu date scrise în cod (cu `224` în loc de `225` din enunț), iar în `04f1702` cu o variantă prin inserare: `inserare` are `v[i] = v[i + 1]` și `dim--`, `cautare` e `int` fără `return`, `prime` și `poz` sunt nefolosite și nu există niciun `cout`. Rulat cu `main` din proiect: ieșire goală, chiar și cu `7 1125 2 314 101 37 225 12` la intrare.

DE CE: `inserare` mută elementele de la dreapta spre stânga citind `v[i + 1]`, deci copiază peste fiecare poziție valoarea de după ea. Bucla merge descrescător, de la `dim` la `poz + 1`, iar prima valoare copiată e `v[dim + 1]`, adică un `0`. Zeroul acesta e tras apoi din poziție în poziție până la `poz + 1`. Rulat: după primul prim găsit (`2`), tabloul devine `1125 0 0 0 0 0 0`. Mai departe `f(0)` întoarce `0` (bucla `d * d <= 0` nu pornește), deci `f(0) == 0` și zeroul e luat drept „prim”. Iar `dim--` scurtează chiar `n`-ul buclei din `solutie4`: 7 → 6 → 5 → 4.

O inserare deplasează la DREAPTA cu `v[i] = v[i – 1]` și CREȘTE dimensiunea, exact cum ai scris-o corect în `varianta73.h:27`. Aici ai scris aceeași funcție a doua oară, pe dos.

FIX: Repune `solutie4` din `4c9f520` (citire cu `cin`, `prime[ct_prime++]`, sortare, `NU EXISTA`); acolo era corectă. Dacă vrei varianta cu inserare, folosește un al doilea tablou `prime` care pornește cu `ct = 0` și inserarea din `varianta73.h` (`v[i] = v[i – 1]`, `d++`). Nu insera în tabloul pe care îl parcurgi. Șterge `cautare` goală sau dă-i `return`.

Repro: `clang++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined varianteBac.cpp` (include `varianta5.h`) → ieșire goală, exit 0; warning `non-void function does not return a value` la `:59`. Trace pe `solutie4`: `i=1 prim 2 poz=0 n 7→6 v: 1125 0 0 0 0 0 0`, `i=2 prim 0`

`n 6→5`, `i=3 prim 0 n 5→4`. Versiunea veche (`git show 4c9f520:varianteBac/varianta5.h`): `7 1125 2 314 101 37 225 12` → `2 37 101`; `4 4 6 8 9` → `NU EXISTA`.

Același bug la:

- `varianta5.h:57–59`: `int cautare(int x[], int n, int elem) { }` fără `return`, UB la apel (singura funcție non-void fără `return` din tot repo-ul, confirmat cu `-Wreturn-type`).
- `varianta5.h:61–76`: `solutie4()` nu afișează nimic; lipsesc afișarea numerelor prime și mesajul `NU EXISTA`. Singurul `varianta*.h` cu 0 `cout`/`printf` în afara comentariilor.
- `varianta5.h:37–43`: `inserare` nu inserează: `v[i] = v[i + 1]` mută spre stânga (la inserare ar fi `v[i] = v[i - 1]`, ca în `varianta73.h:27–33`), `dim--` în loc de `dim++`, lipsește `v[poz] = elem`, parametrul `elem` e nefolosit; la `dim = 100` ar citi `v[101]`.
- `varianta5.h:46–55`: `pozitie` are condiția inversată (`elem > v[i]` în loc de `v[i] > elem`, cf. `varianta73.h:39–49`), iar `poz = 0` implicit pune elementul la început când nu găsește.
- `varianta5.h:68–72`: bucla `for` modifică `n` prin `inserare(v, n, ...)` în timp ce iterează pe `i < n`, iar `v` e folosit și ca sursă, și ca destinație; ar trebui un tablou separat `prime[]` cu dimensiunea lui.
- `varianta5.h:64–65`: `prime` și `poz` declarate dar nefolosite (`-Wunused-variable`); `int poz` de la `:70` le ascunde pe cea de la `:65`.
- `varianta5.h:62`: tabloul conține `224` în loc de `225` din enunț (`:33–34`, exemplul „1125, 2, 314, 101, 37, 225, 12"), deci datele de test nu corespund exemplului.
- `varianta5.h:6`: marcajul `//?? de completat!!` confirmă că fișierul a rămas neterminat.
- `varianta5.h:27–32`: subpunctul b) (citire `n`, citire șir, apeluri utile la `f`, afișare crescătoare, „NU EXISTA") nu este implementat: nu există citire de la tastatură (`cin`) în fișier.

B3 — `varianta3.h`: cifra se pune o singură dată, indiferent de câte ori apare (B1 din runda 4, neatins)

`varianta3.h:69`

CUM E ACUM: Fișierul nu e modificat de la runda 4. `if (f[i] != 0)` pune o pereche per cifră. Exemplul `21523531` dă corect `53211235`, dar `11111111` → `10000001`, iar `11112222` → `21000012`.

DE CE: `f[i]` e numărul de apariții, iar `if` îl folosește doar ca da/nu. Fiecare cifră prezentă primește o pereche (una în față, una în spate), oricâte apariții ar avea. În exemplu fiecare cifră apare exact de 2 ori: o pereche le acoperă pe toate, așa că greșeala nu se vede. La `11111111` cifra 1 are 8 apariții,

adică 4 perechi; codul pune una, iar cele 6 poziții din mijloc rămân `0`, deci `10000001`. Prezență în loc de multiplicitate, același tipar ca în `victor-vectori`.

FIX: în locul `if`-ului:

```
for (int k = 0; k < f[i] / 2; k++) {
    nou = nou + i * (fata + spate);
    fata /= 10;
    spate *= 10;
}
```

(Fragmentul exact e în tabelul Before/After al rundei 4, rămas valabil.)

Repro: copie în scratchpad cu `int nr = 11111111;` → `10000001` (trebuia `11111111`); cu `11112222` → `21000012` (trebuia `22111122`); originalul → `53211235`.

Același bug la:

- Nu am găsit alt loc cu aceeași regulă încălcată în repo. Am verificat prin grep toate `varianteBac/*.h` și `varianteBac.cpp` pentru tiparele de construire a unui număr din frecvențe (`f[i] != 0`, `fr[`, `cnt[`, `]++`, `palindrom`, `cif(`). În afară de `varianta3.h`, doar `varianta14.h` folosește vector de frecvențe (`:16-28`), și acolo e corect: afișează cifra de `fr[i]` ori, într-o buclă `for j < fr[i]`. `varianta95.h:25` (`a[i] > 0`) e doar numărare de pozitive, fără legătură.
- Tot în `varianta3.h`, aceeași funcție `solutie()`, defect de altă natură: linia 51, `int fata = 10000000;`, e codată fix pentru 8 cifre; iar `int nr = ...` e hardcodat la linia 44, în loc de `cin >> n`. Cerința citește `n` de la tastatură, deci e abatere de la enunț, nu repetiția acestui bug.
- Singura locație de reparat este `varianta3.h:68-75`: bucla `for (int i = 9; i >= 0; i--)` cu `if (f[i] != 0)` trebuie să facă `f[i] / 2` iterații interioare, cu avansarea `fata` / `spate` la fiecare pereche.

B4 — `varianta80.h`: `NU EXISTA` nu se poate afișa, iar ordinea crescătoare lipsește (B3 din runda 4, neatins)

`varianta80.h:18`

CUM E ACUM: Nemodificat de la runda 4. Pe exemplu iese `44 111 7 5` în loc de `5 7 44 111`. Pe `{30,25,12}` nu iese nimic, deși trebuia `NU EXISTA`.

DE CE: `vf` încearcă să răspundă la două întrebări: „numărul ĂSTA are cifre egale?" și „am găsit VREUNUL?". `vf = 1` la începutul fiecărui pas răspunde la prima, dar șterge răspunsul la a doua. După primul pas `vf` nu mai e niciodată `-1`, deci ramura `NU EXISTA` e moartă. Pe `{30,25,12}` bucla se termină cu `vf = 0` și nu se afișează nimic. Ordinea lipsește pentru că numerele se afișează pe loc. Ca să le sortezi, trebuie mai întâi să le păstrezi într-un tablou `b`, iar `k == 0` pe acel tablou înlocuiește steagul.

FIX: Codul din FIX-ul B3 al rundei 4: `b[k++] = a[i]` pentru numerele cu cifre egale, sortare pe `b[0..k)`, `if (k == 0) cout << "NU EXISTA";`, apoi afișare. Citirea din `bac.in` cu `while (fin >> a[d]) d++;`.

Repro: main cu `solutie()` din `varianta80.h` → `44 111 7 5`. Copie în scratchpad cu `{ 30,25,12 }` și `d = 3` → ieșire goală `[]`.

Același bug la:

- `varianteBac/varianta5.h:27–31` (cerința b: prime în ordine crescătoare, altfel `NU EXISTA`) nu este implementată. `solutie4()` nu afișează nimic și nu are mesajul `NU EXISTA`; `inserare`, `cautare` și `pozitie` sunt greșite (detalii în B2).
- `varianteBac/varianta80.h:15–35` nu face nicio sortare (cerința „în ordine crescătoare"). Se rezolvă prin colectarea numerelor într-un tablou, sortare și afișare, sau prin sortarea lui `a` înainte de buclă; astfel `NU EXISTA` se decide după buclă dintr-un contor/flag setat doar la găsire (ca în `varianta25.h:19–30` sau `tema-exercitii-doi-indici.h:117`).

🟡 Importante

- **M1** — `varianta24.h:78` : tot `4, 8 ,5.667` ; enunțul cere doar media. Linia 5 tot `#include <iomanip>;` (runda 4 M1, neatins, rulat). Linia 78 trebuie să fie `cout << fixed << setprecision(3) << medie;` .
- **M2** — `varianta8.h:153–168` : `solutie4` cheamă tot `sortare` ×2 + `interclasare` ; `sub` e corect, dar neapelat (runda 4 M2, neatins). `solutie4` devine `sub(a, 5, b, 3);` .
- **M3** — `varianteBac.cpp` : proiectul compilează 1 header din 46 (acum `varianta5.h`). Costul concret în runda asta: `varianta5` și `varianta73` au fiecare câte un `inserare` , cu aceeași semnătură → `redefinition` dacă le incluzi împreună; `cautare` are două semnături diferite.
- **M4** — zero `fstream` în 46 de fișiere (a cincea rundă). În plus, regres: `varianta5` a pierdut `cin` în `eb45ac6` , iar datele scrise în cod au `224` în loc de `225` din enunț. `varianta73` c) cere `sir.in` / `sir.out` , câte 10 pe rând: nescris.
- **M5** — `varianta67.h:26` : `d <= aux / 2` (runda 4 M5, neatins). Corect: `d * d <= aux` .
- **M6** — `varianta92.h:45` : `cout << endl;` înainte de prima linie (runda 4 M6, neatins).
- **M7** — coliziuni de nume, agravat: acum și `inserare` ×2, `cautare` ×2 → `namespace vNN { }` în fiecare header.
- **M8** — `varianta73.h:74` : `crescator` (bubble sort cu steag) e corectă, dar nefolosită; enunțul cere sortarea prin apeluri de `Cautare` .
- **M9** — restanțe din runda 3, neatinse: `reuniune` fără `p = 0` , `varianta42` sortează tabloul apelantului, prompturi `n=` în `varianta1/6/11` , `varianta46` `k` necitit, `varianta55.h:33` `int n;` .

🟢 Cleanups

- **C1** — `varianta23.h:31` : `i < n` citește `x[n]` ; corect `i < n - 1` . **A patra rundă.**
- **C2** — `varianta26.h:9` : `//??` adăugat pe un fișier închis corect în runda 2 (`10 2 19 9 15`); marcajul nu mai spune unde e lucrul nefăcut.
- **C3** — `varianta5.h:64-65,70` : `prime` nefolosit, `poz` declarat de două ori (cel din buclă îl ascunde pe cel de afară); `:37` `elem` nefolosit.
- **C4** — `varianta3.h:60` `int p = 1;` și `varianta24.h:60` `int ct = 1;` nefolosite.
- **C5** — `varianta73.h:35-36` : comentariile de lucru `// 23 ,34,56, 100 85` rămase în cod.

Before / After

#	fișier	acum	trebuie
B1	`varianta73.h:39`	`void cautare(...)` + `gasitPozitie` liniar, `int poz = 0;` , `n` prin valoare, `n = 11` la start	`int cautare(int n, int x[], int v) { int inf = 1, sup = n; while (inf <= sup) { int mij = (inf + sup) / 2; if (x[mij] > v) sup = mij - 1; else inf = mij + 1; } return inf; }` , `n = 0` la start, `x[poz] = val; n++;`
B2	`varianta5.h:37–43`	`v[i] = v[i + 1]` + `dim--`	`v[i] = v[i - 1]` + `dim++` + `v[poz] = elem` (ca în `varianta73.h:27`)
B2	`varianta5.h:61–76`	`solutie4` fără `cout` , insert în tabloul parcurs	`solutie4` din `4c9f520` : `cin` , `prime[ct_prime++]` , sortare, `NU EXISTA`
B2	`varianta5.h:57–59`	`int cautare(...) { }`	șters, sau cu `return`
B3	`varianta3.h:69`	`if (f[i] != 0) {`	`for (int k = 0; k < f[i] / 2; k++) {`
B4	`varianta80.h:18–38`	`vf` resetat la fiecare număr, afișare pe loc	`b[k++] = a[i]` , sortare, `if (k == 0) NU EXISTA`

✅ Ce e corect — verificat prin rulare

- `varianta73.h:27` `inserare`: deplasare spre dreapta cu `v[i - 1]`, `d` prin referință, `d++`. Confirmat prin rulare cu poziția dată. Trebuie doar apelată cu o poziție corectă și cu `n` real.
- `varianta5.h:18` `f` cu `d * d <= a`: `f(2) == 2`, `f(1125) == 3`.
- `varianta5.h`, versiunea `4c9f520`: corectă pe ambele cazuri (`2 37 101`, `NU EXISTA`). Ai avut-o.
- Poarta de compilare: 46/46 headere trec `g++ -std=c++17 -fsyntax-only`.

Q&A — răspunde-mi la astea trei

1. `varianta73.h`: `gasitPozitie(x, 11, 700)` întoarce `0`. Ce înseamnă `poz = 0` când bucla nu găsește niciun element mai mare decât `v`, și ce valoare ar trebui să aibă în cazul ăsta? Apoi: de ce `d++` din `inserare` nu schimbă `n`-ul din `solutie()`?
2. `varianta5.h`: urmărește `inserare(v, n, 0, 2)` pe `{1125,2,314,101,37,224,12}`, cu `n = 7`. Ce valoare ajunge în `v[1]` după buclă și de unde vine? De ce `f(0) == 0` face programul să ia zeroul drept număr prim?
3. Căutarea din `varianta73` a) trebuie să fie „eficientă din punct de vedere al timpului". Pe 1000 de elemente, câte comparații face cel mult varianta ta liniară și câte face căutarea binară? Ce scrii la punctul b)?

Ce ai de făcut, în ordine

Regula rundei: înainte să rescrii un fișier care mergea, rulează-l pe exemplul din enunț și pe unul inventat de tine. Dacă iese corect, nu-l atinge. Când ți se dă un fix de o linie, aplică exact linia aia.

1. `varianta5.h` — repune `solutie4` din `4c9f520` (`git show 4c9f520:varianteBac/varianta5.h`) (B2).
2. `varianta73.h` — înapoi la căutarea binară cu `return inf;`, `n = 0` la start, `n` real în program (B1); apoi b) și c).
3. `varianta3.h` — `if`-ul devine `for` până la `f[i] / 2` (B3).
4. `varianta80.h` — tabloul `b`, sortarea, `k == 0` (B4).
5. Într-o linie fiecare: M1, M2, M5, M6, C1.
6. O dată pentru tot repo-ul: `namespace vNN { }` (M7) și citirea din fișier (M4).

Runda 4 — 2026-10-06 — sha `4c9f520`

Trei commit-uri (`95205bb` , `936eea4` , `4c9f520`), 21 de fișiere, 14 variante noi (67, 73, 80–86, 91–95). Poarta de compilare: trecută, 46/46. 11 din 14 variante noi corecte. Tiparul rundei: exemplul din enunț e singurul test.

#	constatare	fișier	stare la runda 5
B1	cifra se pune o dată, oricâte apariții are (`11111111` → `10000001`)	`varianta3.h:68–75`	🔴 **neatins** → B3 runda 5
B2	`cautare` întoarce `-1` pe orice intrare; `sup = n + 1` iese din tablou	`varianta73.h:16–30`	🔴 **rescris, tot greșit** (liniar, `void`) → B1 runda 5
B3	`NU EXISTA` mort (`vf` resetat) + ordine lipsă	`varianta80.h:18–38`	🔴 **neatins** → B4 runda 5
M1	`4, 8 ,5.667` + `#include <iomanip>;`	`varianta24.h:5,78`	🔴 neatins → M1 runda 5
M2	`solutie4` rulează codul vechi, `sub` neapelat	`varianta8.h:153–168`	🔴 neatins → M2 runda 5
M3	proiectul compilează 1 fișier din 46	`varianteBac.cpp`	🔴 neatins → M3 runda 5
M4	zero `fstream` (a patra rundă)	tot repo-ul	🔴 neatins + regres v5 → M4 runda 5
M5	`d <= aux / 2` (1,13 s pe 3 prime de 9 cifre)	`varianta67.h:26`	🔴 neatins → M5 runda 5
M6	linie goală înaintea ieșirii	`varianta92.h:45`	🔴 neatins → M6 runda 5
M7	15× `solutie` , 18× `solutie3`	tot repo-ul	🔴 agravat → M7 runda 5

#	constatare	fișier	stare la runda 5
—	restanțe din runda 3 (`reuniune` , v42, prompturi, v46, v55)	diverse	🔴 neatinse → M9 runda 5
C1	`shift` citește `x[n]`	`varianta23.h:31`	🔴 neatins → C1 runda 5
C2	marcaje `//??` pe fișiere corecte	diverse	🔴 agravat → C2 runda 5
C3	`int f[100]` , `int p = 1;` nefolosit	`varianta3.h:46,60`	🔴 neatins → C4 runda 5
C4	`int ct = 1;` nefolosit	`varianta24.h:60`	🔴 neatins → C4 runda 5
C5	variabilă `max` peste `std::max`	`varianta86.h:18`	🔴 neatins

Închise din runda 3 (confirmate în runda 4): `doarInA` cu `NU EXISTA` , `alKlea` cu `return -1` , `sub` corect pe 7/7. Corecte pe nume: `varianta84` (`% 2 != 0` pe negative), `varianta83` (`abs` înainte de cifre), `varianta91` (garda `n == 0`), `varianta86` (două maxime într-o parcurgere).

Runda 3 — 2026-09-29 — sha `dad5251`

Un commit (`dad5251`), 13 fișiere, 511 linii. Poarta de compilare: trecută, 32/32. Fișa `exercitii-doi-indici` rezolvată integral, 19/20 exemple corecte, iar mecanismul a migrat înapoi în `varianta8.h`. Tiparul rundei: **review-ul devine comentariu** — fix-ul copiat ca `//!` în `varianta24.h`, codul neschimbat.

#	constatare	fișier	stare la runda 4
B1	fix-ul din runda 2 scris ca comentariu	`varianta24.h`	🟡 pe jumătate — `5.667` ✅, `mini` / `maxi` încă afișate → M1 runda 4
B2	`doarInA` fără `NU EXISTA`	`tema…doi-indici.h:172`	✅ **închis**
B3	palindromul nu se construiește	`varianta3.h:70`	🔴 rescris, tot greșit pe cifre repetate → B1 runda 4
B4	`sub`, cerut de enunț, nu există	`varianta8.h`	✅ **închis** — corect pe 7/7 · neapelat → M2 runda 4
B5	`alKlea` fără `return`	`tema…doi-indici.h:423`	✅ **închis**
M1	proiectul compilează 1 fișier din 32	`varianteBac.cpp`	🔴 agravat, 1 din 46 → M3 runda 4
M2	`reuniune` afișează și nu resetează `p`	`tema…doi-indici.h`	🔴 neatins
M3	zero `fstream`	tot repo-ul	🔴 neatins → M4 runda 4
M4	`p` / `sortare` distrug tabloul apelantului	v42, v8	🔴 neatins
M5	18× `solutie3`	tot repo-ul	🔴 agravat → M7 runda 4
M6	prompturi la tastatură	v1, v6, v11	🔴 neatins
M7	`k` necitit	`varianta46.h`	🔴 neatins

#	constatare	fișier	stare la runda 4
M8	`int n;` necitit	`varianta55.h:33`	🔴 neatins
C1	`shift` citește `x[n]`	`varianta23.h:31`	🔴 neatins → C1 runda 4
C2–C6	variabile nefolosite, marcaje, nume	diverse	🔴 neatinse

Runda 2 — 2026-09-23 — sha `02640ce`

Patru commit-uri (`a9c3abc`, `d068314`, `7bbed5c`, `02640ce`). 16 variante noi, 5 atinse. Poarta de compilare: trecută, 30/30 headere, warning-uri în 4 fișiere din care 3 bug-uri reale.

Verdict: algoritmii s-au făcut buni — 13 din 16 variante noi dau răspunsul corect. Tiparul rundei: *rezolvi altă problemă decât cea cerută* — subprogramul cerut ocolit, datele nu vin din sursa indicată, formatul de ieșire ignorat. Vechiul tipar („te oprești la un pas de final") s-a concentrat în trei `if`-uri cu corpul gol.

#	constatare	fișier	stare la runda 3
B1	nu afișează nimic; `cif` definit și apelat de 0 ori	`varianta3.h`	🟡 pe jumătate — `cif` apelat ✅, palindromul lipsă → B3 runda 3
B2	`sub` tot gol; `interclasare` pierde elemente pe `a={2,4} b={1}`	`varianta8.h`	🟡 pe jumătate — algoritm ✅ **corect pe 4/4**, `sub` inexistent → B4 runda 3
B3	`4, 8 ,5.66667` în loc de `5.667`	`varianta24.h`	🔴 **neatins** — fix-ul scris ca comentariu → B1 runda 3
B4	mesajul `nu exista numere prime`	`varianta5.h`	✅ **închis** — `NU EXISTA` + citire de la tastatură
B5	`f` schelet: `if` gol, `int k` nu `int &k`, `int` fără `return`	`varianta25.h`	✅ **închis** — dă `24`
B6	un `0` în plus la final; trei convenții de indexare	`varianta26.h`	✅ **închis** — `10 2 19 9 15`
B7	`i < d` în loc de `i <= d`	`varianta52.h`	✅ **închis** — dă `4`
M1	9 variante nu citesc din sursa cerută; zero `fstream`	6 enunțuri	🔴 neatins → M3 runda 3
M2	coliziuni de nume (17× `solutie3`)	tot repo-ul	🔴 agravat (18×) → M5 runda 3
M3	`p` sortează in-place tabloul apelantului	`varianta42.h`	🔴 neatins → M4 runda 3

#	constatare	fișier	stare la runda 3
M4	`k` nu se citește, tabloul nu se construiește	`varianta46.h`	🔴 neatins → M7 runda 3
M5	`ordin(0)` dă 1	`varianta55.h`	✅ **închis** — garda pusă · ⚠️ exemplul meu (`5` → `50`) era greșit, vezi runda 3
M6	prompturi la tastatură	6 locuri	🔴 neatins → M6 runda 3
M7	`nule` afișează, deși n-ar trebui	`varianta20.h`	✅ **închis** — afișarea mutată în `solutie3`
C1	`shift` citește `x[n]` , în afara zonei permutate	`varianta23.h:31`	🔴 neatins → C1 runda 3
C2	`count` peste `std::count`	`varianta18.h:18`	🔴 neatins
C3	variabile declarate și nefolosite, 5 locuri	v24, v3, v25, v8	🟡 4 din 5 dispărute cu rescrierile; rămâne `ct`
C4	`int f(long a)` — îngustare tăcută	`varianta5.h:27`	🔴 neatins
C5	`<cmath>` nefolosit în toate fișierele; `Header.h` orfan	tot repo-ul	🔴 neatins
C6	marcaje `//??` rămase	5 fișiere	🟡 2 din 5 sunt acum pe fișiere corecte → C4 runda 3
C7	spațiu în plus la final de linie	9 fișiere	🔴 neatins

#	constatare	fișier	stare la runda 3
C8	`stergere` citește `x[dim]`	`varianta24.h:49`	🔴 neatins
C9	`a[i] % k` cu `k = 0` ; `fr[cif]++` fără gardă	v16, v14	🔴 neatins

Trei 🔴 închise curat, două pe jumătate. Cele închise sunt exact cele unde primise linia de cod; cele pe jumătate sunt acolo unde constatarea avea două lucruri în ea.

Corecte la runda 2, spuse pe nume: inversarea prin apeluri repetate de `shift` (`varianta23`), Euclid în lanț (`varianta66`), `ordin()` izolat ca subprogram (`varianta55`), două contoare cu ritmuri diferite în `fibo` (`varianta62`).

Livrate în aceeași rundă: `exercitii-runda-2.pdf` (8 exerciții, câte unul per mecanism greșit) și `exercitii-doi-indici.pdf` (7 exerciții pe mecanismul din `varianta8.h` , cu constrângerea „o singură parcurgere"). Pe a doua a rezolvat-o integral — vezi runda 3.

Runda 1 — 2026-09-15 — sha `46a53db`

Commit `46a53db tema` (variantele 3, 5, 24, 39) plus verificarea integrală a celor din `4281854 1st push`. Poarta de compilare: trecută, 14/14 headere, 4 warning-uri din care 3 bug-uri reale.

Verdict: din cele 4 fișiere ale temei, unul singur (`varianta39`) producea răspunsul cerut. Tiparul rundei: *te oprești la un pas de final* — algoritmul scris, linia care îl leagă de răspuns lipsă.

#	constatare	fișier	stare la runda 2
B1	parametrii de ieșire transmiși prin copie, nu prin referință	`varianta24.h:18`	✅ închis
B2	`for (int i = i; ...)` — bucla pornește de la o valoare nedefinită	`varianta24.h:23`	✅ închis
B3	nu afișează nimic; media s-ar calcula în întregi	`varianta24.h:57`	🟡 pe jumătate → B3 runda 2
B4	cifra 9 nu e numărată niciodată (`i < 9` pe un tablou de 10)	`varianta3.h:48,62`	⚪ dispărut odată cu rescrierea
B5	palindrom: `0` nu se afișează, zeroul lipsește, mijlocul lipsește	`varianta3.h:50–66`	🔴 înrăutățit → B1 runda 2
B6	punctul b) nu e scris: `f` nu e apelat niciodată; mesaj greșit	`varianta5.h:44`	🟡 pe jumătate → B4 runda 2
B7	funcție `int` care nu întoarce nimic	`varianta39.h:15`	✅ închis
B8	`sub` e schelet: interclasează și nu afișează nimic	`varianta8.h:18–33`	🔴 neatins → B2 runda 2
M1	două variante nu pot exista în același proiect (nume duplicate)	tot repo-ul	🔴 agravat → M2 runda 2

#	constatare	fișier	stare la runda 2
M2	`sort` definit de două ori și suprapus peste `std::sort`	`varianta1.h:44`, `varianta12.h:28`	🔴 neatins
M3	prompturile de la tastatură strică evaluarea automată	5 locuri	🔴 neatins → M6 runda 2
M4	`varianta39` cere citire din `bac.in`, nu din `cin`	`varianta39.h:21`	🔴 neatins → M1 runda 2
M5	subprogramul `nule` afișează, deși n-ar trebui	`varianta20.h:29–31`	🔴 neatins → M7 runda 2
C1	`a[i] % k` cu `k = 0`	`varianta16.h:19`	🔴 neatins
C2	`fr[cif]++` fără gardă pe interval	`varianta14.h:22`	🔴 neatins
C3	`int f(long a)` — îngustare tăcută pe `return a`	`varianta5.h:32`	🔴 neatins → C4 runda 2
C4	`<cmath>` nefolosit peste tot; `Header.h` orfan	tot repo-ul	🔴 neatins → C5 runda 2
C5	marcaje `//??` rămase	5 fișiere	🔴 neatins → C6 runda 2

Corecte la runda 1, verificate prin rulare: `varianta1.h` (b), `varianta7.h`, `varianta12.h`, `varianta16.h`, `varianta17.h`, `varianta20.h`, `varianta39.h`, plus `varianta6.h`, `varianta11.h`, `varianta14.h` fără obiecții de fond.